

Nama : Elyza Putri Rahayu

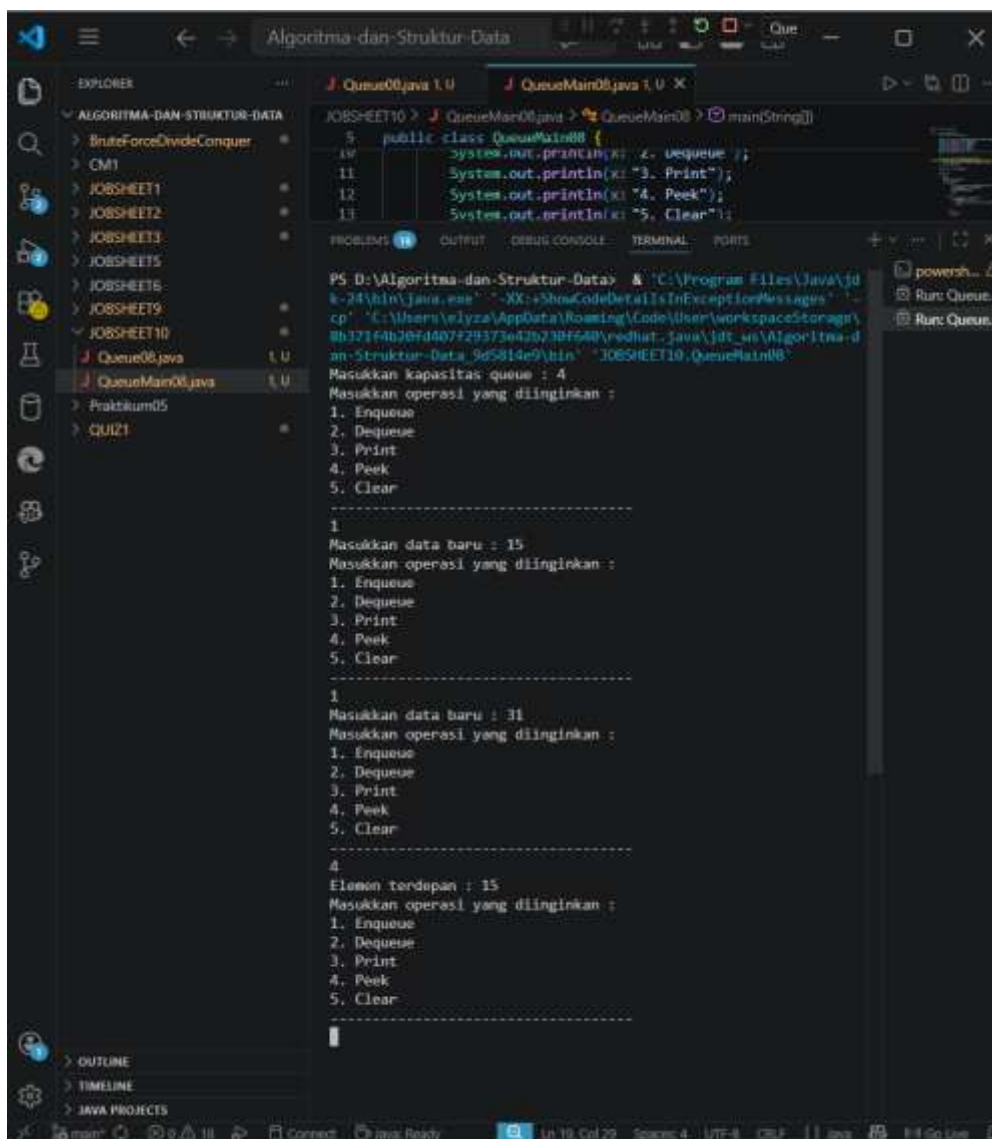
NIM : 2541007020035

Kelas : TI – 1G

No Absen : 08

JOBSHEET 10 QUEUE PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

PERCOBAAN 1



The screenshot shows an IDE with a project named "Algoritma-dan-Struktur-Data". The Explorer panel on the left shows the project structure, including a folder "JOBSHEET10" and a file "QueueMain08.java". The main editor displays the code for "QueueMain08.java", which implements a queue using an array. The code includes methods for enqueue, dequeue, print, peek, and clear. The output window shows the execution of the program, where the user enters a capacity of 4, followed by several operations (enqueue, dequeue, print, peek, clear) and data values (15, 31). The output also shows the element at the front of the queue (15) and the element at the back (31).

```
JOBSHEET10 > QueueMain08.java > QueueMain08 > main(String[])
5 public class QueueMain08 {
10     >system.out.println(x); < dequeue >
11     >system.out.println(x); < 3. Print >
12     >system.out.println(x); < 4. Peek >
13     >system.out.println(x); < 5. Clear >

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Algoritma-dan-Struktur-Data> & 'C:\Program Files\Java\jdk-24\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\elyza\AppData\Roaming\Code\User\workspaceStorage\0b37164b20fd407f29373e42b23ff640\redhat-java\jdk_24\Algoritma-dan-Struktur-Data_9d5814e9\bin" "JOBSHEET10.QueueMain08"
Masukkan kapasitas queue : 4
Masukkan operasi yang diinginkan :
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru : 15
Masukkan operasi yang diinginkan :
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru : 31
Masukkan operasi yang diinginkan :
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
4
Elemen terdapan : 15
Masukkan operasi yang diinginkan :
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
```

Kode Program Queue08 :

```
package P1JOBSHEET10;
```

```
import java.util.Queue;
```

```
public class Queue08 {
```

```
    int[] data;
```

```
    int front;
```

```
    int rear;
```

```
    int size;
```

```
    int max;
```

```
    public Queue08(int n) {
```

```
        max = n;
```

```
        data = new int[max];
```

```
        size = 0;
```

```
        front = rear = -1;
```

```
    }
```

```
    public boolean IsEmpty() {
```

```
        if (size == 0) {
```

```
            return true;
```

```
        } else {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    public boolean IsFull() {
```

```
        if (size == max) {
```

```
        return true;
    } else {
        return false;
    }
}
```

```
public void peek() {
    if (!IsEmpty()) {
        System.out.println("Elemen terdepan : " + data[front]);
    } else {
        System.out.println("Queue masih kosong");
    }
}
```

```
public void print() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.println(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}
```

```
public void clear() {
    if (!IsEmpty()) {
```

```

        front = rear = -1;

        size = 0;

        System.out.println("Queue berhasil dikosongkan");
    } else {

        System.out.println("Queue masih kosong");

    }

}

```

```

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
    }
}

```

```

        data[rear] = dt;

        size++;

    }

}

```

```

public int Dequeue() {
    int dt = 0;

    if (IsEmpty()) {

```

```

        System.out.println("Queue masih kosong");
    } else {
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
            } else {
                front++;
            }
        }
    }
    return dt;
}
}

```

Kode program QueueMain08 :

```
package P1JOBSHEET10;
```

```
import java.util.Scanner;
```

```
public class QueueMain08 {
```

```
    public static void menu() {
```

```
        System.out.println("Masukkan operasi yang diinginkan : ");
```

```
        System.out.println("1. Enqueue");
```

```
        System.out.println("2. Dequeue");
```

```
        System.out.println("3. Print");
```

```
System.out.println("4. Peek");
System.out.println("5. Clear");
System.out.println("-----");
}
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Masukkan kapasitas queue : ");
    int n = sc.nextInt();

    Queue08 Q = new Queue08(n);

    int pilih;
    do {
        menu();
        pilih = sc.nextInt();
        switch (pilih) {
            case 1:
                System.out.print("Masukkan data baru : ");
                int dataMasuk = sc.nextInt();
                Q.Enqueue(dataMasuk);
                break;

            case 2:
                int dataKeluar = Q.Dequeue();
                if (dataKeluar != 0) {
                    System.out.println("Data yang dikeluarkan : " + dataKeluar);
                    break;
                }
        }
    }
```

```

        case 3:
            Q.print();
            break;

        case 4:
            Q.peek();
            break;

        case 5:
            Q.clear();
            break;
    }
} while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);
}
}

```

Pertanyaan :

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0 ?
 Jawab : Pada saat queue pertama kali dibuat, kondisi queue masih kosong, sehingga belum ada data yang masuk. Oleh karena itu, nilai front dan rear diinisialisasi dengan -1 sebagai penanda bahwa belum ada elemen yang tersimpan di dalam queue. Sedangkan size bernilai 0 karena memang jumlah data dalam queue masih nol. Ketika data pertama masuk, barulah nilai front dan rear akan berubah menjadi 0.
2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!
 if (rear == max - 1) { rear = 0;
 Jawab : Potongan kode ini digunakan untuk mengatur pergerakan indeks rear dalam konsep **circular queue (antrian melingkar)**.
 Artinya, ketika posisi rear sudah berada di indeks terakhir array (max - 1), maka posisi tersebut akan dikembalikan ke indeks awal yaitu 0.
 Hal ini dilakukan agar ruang dalam array dapat digunakan kembali tanpa harus menggeser data yang sudah ada.
3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!
 if (front == max - 1) { front = 0;

jawab : Kode ini berfungsi untuk mengatur posisi front saat melakukan proses pengambilan data (dequeue).

Jika front sudah berada di indeks terakhir array, maka nilainya akan dikembalikan ke indeks 0.

Tujuannya sama seperti pada rear, yaitu agar queue dapat berjalan secara melingkar dan tetap efisien dalam penggunaan memori.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?

Jawab : Perulangan pada method print dimulai dari front karena posisi data pertama dalam queue tidak selalu berada di indeks 0.

Hal ini disebabkan oleh proses enqueue dan dequeue yang membuat posisi front bisa bergeser.

Jika perulangan dimulai dari 0, maka data yang sudah keluar dari queue bisa ikut tercetak kembali, sehingga hasilnya menjadi tidak sesuai. Oleh karena itu, perulangan harus dimulai dari posisi front.

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

jawab : Kode ini digunakan untuk menggeser indeks secara melingkar.

Setiap kali indeks i ditambah 1, hasilnya akan di-modulo (%) dengan nilai max.

Jika indeks sudah mencapai batas akhir array, maka akan kembali ke awal (indeks 0).

Dengan cara ini, pergerakan indeks tidak akan keluar dari batas array.

6. Tunjukkan potongan kode program yang merupakan queue overflow!

Jawab : Queue overflow terjadi ketika queue sudah penuh tetapi masih dilakukan penambahan data.

Hal ini ditandai dengan kondisi berikut:

```
if (IsFull()) {  
    System.out.println("Queue sudah penuh");  
}
```

Kondisi tersebut menunjukkan bahwa queue tidak dapat menerima data baru karena kapasitasnya sudah maksimum.

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawab :

```
public void Enqueue(int dt) {  
    if (IsFull()) {  
        System.out.println("Queue sudah penuh");  
        System.exit(0);  
    }  
}  
  
public int Dequeue() {
```



```

int dt = 0;
if (IsEmpty()) {
    System.out.println("Queue masih kosong");
    System.exit(0);
}

```

PERCOBAAN 2

The screenshot shows an IDE window titled "Algoritma-dan-Struktur-Data" with a terminal running a Java program. The Explorer panel on the left shows a project structure with files like `Queue08.java`, `QueueMain08.java`, and `AntrianLayanan08.java`. The terminal output shows the program's execution, including menu prompts, user input for menu selection and student details, and the resulting queue state.

```

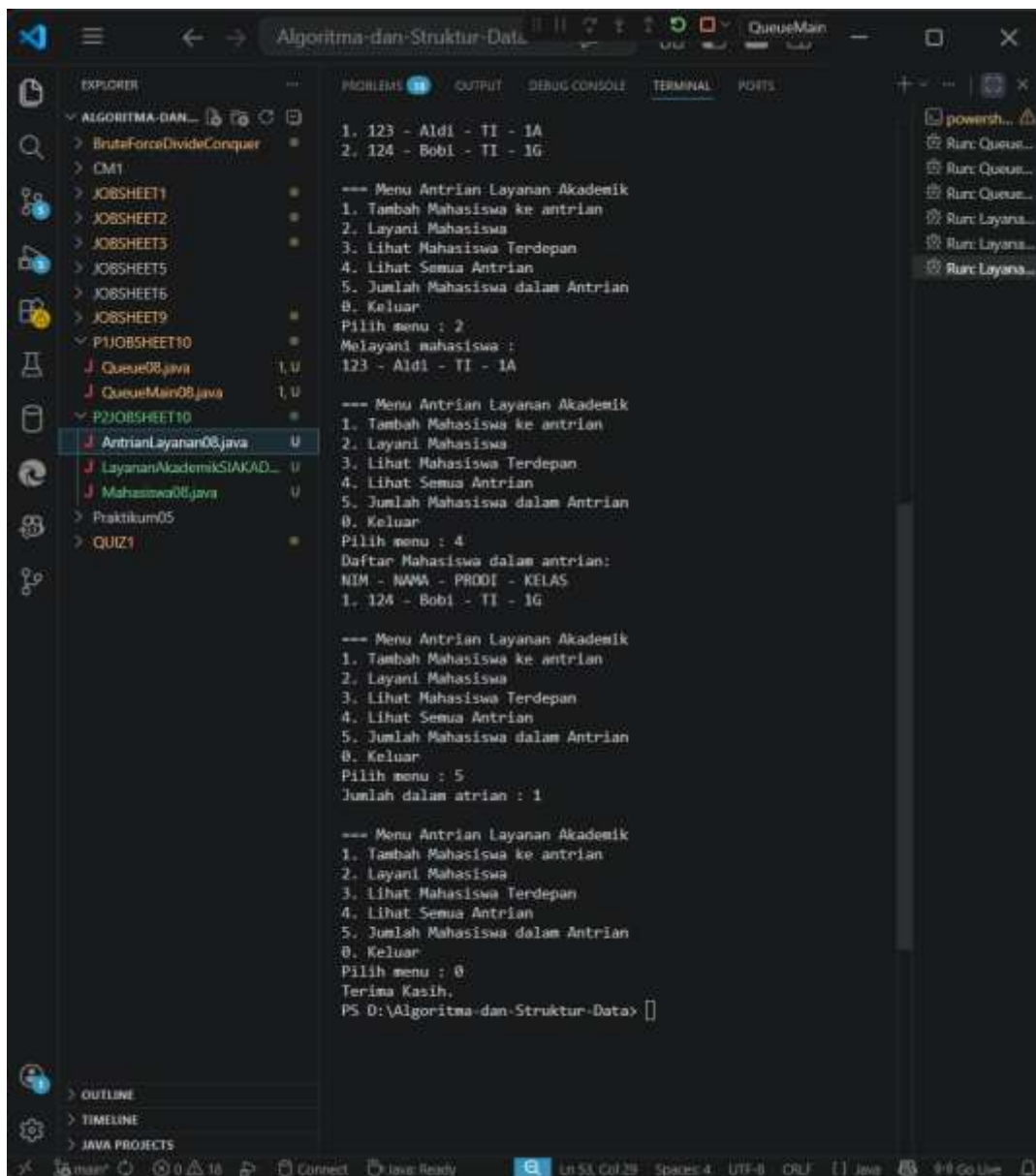
PS D:\Algoritma-dan-Struktur-Data> & 'C:\Program Files\Java\jdk-24\bin\java.exe' ^
-X:+ShowCodeDetailsInExceptionMessages ^
-cp 'C:\Users\elyza\AppData\Roaming\Code\User\workspaceStorage\8b371f4b20fd407f29373e42b230f640\redhat.java\jdt_ws\Algoritma-dan-Struktur-Data_9d5814e9\bin' 'P2\JOBSHEET10\LayananAkademik51A\KAD08'

=== Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu : 1
NIM : 123
Nama : Aldi
Prodi : TI
Kelas : 1A
Aldi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu : 1
NIM : 124
Nama : Bobi
Prodi : TI
Kelas : 1G
Bobi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu : 4
Daftar Mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G

```



Kode Program Mahasiswa08 :

package *P2JOBSHEET10*;

public class Mahasiswa08 {

String nim;

String nama;

String prodi;

String kelas;

```

public Mahasiswa08(String nim, String nama, String prodi, String kelas) {
    this.nim = nim;
    this.nama = nama;
    this.prodi = prodi;
    this.kelas = kelas;
}

public void tampilkanData() {
    System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
}
}

```

Kode Program AntrianLayanan08 :

```

package P2JOBSHEET10;

public class AntrianLayanan08 {
    Mahasiswa08[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan08(int max) {
        this.max = max;
        this.data = new Mahasiswa08[max];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }
}

```

```
public boolean IsEmpty() {  
    if (size == 0) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

```
public boolean IsFull() {  
    if (size == max) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

```
public void lihatTerdepan() {  
    if (!IsEmpty()) {  
        System.out.println("Antrian masih kosong");  
    } else {  
        System.out.println("Mahasiswa terdepan : ");  
        System.out.println("NIM - NAMA - PRODI - KELAS");  
        data[front].tampilkanData();  
    }  
}
```

```
public void tampilkanSemua() {  
    if (IsEmpty()) {  
        System.out.println("Antrian kosong");  
    }  
}
```

```

        return;
    }

    System.out.println("Daftar Mahasiswa dalam antrian: ");
    System.out.println("NIM - NAMA - PRODI - KELAS");
    for (int i = 0; i < size; i++) {
        int index = (front + i) % max;
        System.out.print((i + 1) + ". ");
        data[index].tampilkanData();
    }
}

public int getJumlahAntrian() {
    return size;
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void tambahAntrian(Mahasiswa08 mhs) {
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambah mahasiswa.");
        return;
    }
}

```

```

        rear = (rear + 1) % max;
        data[rear] = mhs;
        size++;
        System.out.println(mhs.nama + " berhasil masuk ke antrian.");
    }

    public Mahasiswa08 layaniMahasiswa08() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong");
            return null;
        }
        Mahasiswa08 mhs = data[front];
        front = (front + 1) % max;
        size--;
        return mhs;
    }
}

```

Kode program LayananAkademikSIKAD08 :

```
package P2JOBSHEET10;
```

```
import java.util.Scanner;
```

```

public class LayananAkademikSIKAD08 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        AntrianLayanan08 antrian = new AntrianLayanan08(5);

        int pilihan;
    }
}

```

```
do {
```

```
    System.out.println("\n=== Menu Antrian Layanan Akademik");
```

```
    System.out.println("1. Tambah Mahasiswa ke antrian");
```

```
    System.out.println("2. Layani Mahasiswa");
```

```
    System.out.println("3. Lihat Mahasiswa Terdepan");
```

```
    System.out.println("4. Lihat Semua Antrian");
```

```
    System.out.println("5. Jumlah Mahasiswa dalam Antrian");
```

```
    System.out.println("0. Keluar");
```

```
    System.out.print("Pilih menu : ");
```

```
    pilihan = sc.nextInt(); sc.nextLine();
```

```
    switch (pilihan) {
```

```
        case 1:
```

```
            System.out.print("NIM : ");
```

```
            String nim = sc.nextLine();
```

```
            System.out.print("Nama : ");
```

```
            String nama = sc.nextLine();
```

```
            System.out.print("Prodi : ");
```

```
            String prodi = sc.nextLine();
```

```
            System.out.print("Kelas : ");
```

```
            String kelas = sc.nextLine();
```

```
            Mahasiswa08 mhs = new Mahasiswa08(nim, nama, prodi, kelas);
```

```
            antrian.tambahAntrian(mhs);
```

```
            break;
```

case 2:

```
Mahasiswa08 dilayani = antrian.layaniMahasiswa08();  
if (dilayani != null) {  
    System.out.println("Melayani mahasiswa : ");  
    dilayani.tampilkanData();  
}  
break;
```

case 3:

```
antrian.lihatTerdepan();  
break;
```

case 4:

```
antrian.tampilkanSemua();  
break;
```

case 5:

```
System.out.println("Jumlah dalam atrian : " + antrian.getJumlahAntrian());  
break;
```

case 0:

```
System.out.println("Terima Kasih.");  
break;
```

default:

```
System.out.println("Pilihan tidak valid");  
}
```

```
} while (pilihan != 0);
```

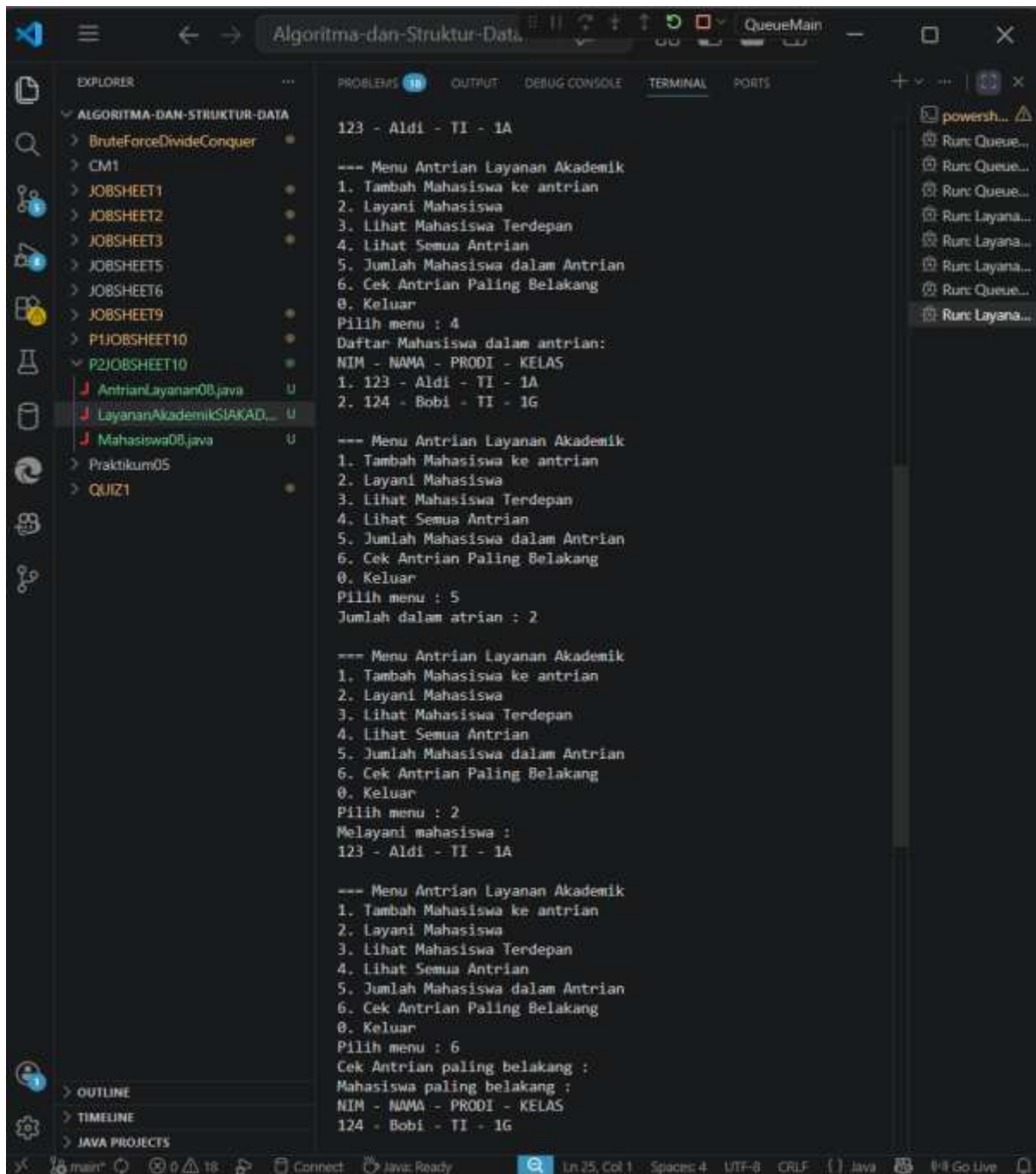


```
        sc.close();  
    }  
}
```

Pertanyaan :

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

Jawab :



```
Algoritma-dan-Struktur-Data QueueMain

EXPLORER
  ALGORITMA-DAN-STRUKTUR-DATA
    BruteForceDivideConquer
    CM1
    JOBSHEET1
    JOBSHEET2
    JOBSHEET3
    JOBSHEET5
    JOBSHEET6
    JOBSHEET9
    P1JOBSHEET10
    P2JOBSHEET10
    AntrianLayanan00.java
    LayananAkademikSIKAD...
    Mahasiswa00.java
    Praktikum05
    QUIZ1

PROBLEMS 18 OUTPUT DEBUG CONSOLE TERMINAL PORTS
123 - Aldi - TI - 1A

--- Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu : 4
Daftar Mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G

--- Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu : 5
Jumlah dalam atrian : 2

--- Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu : 2
Melayani mahasiswa :
123 - Aldi - TI - 1A

--- Menu Antrian Layanan Akademik
1. Tambah Mahasiswa ke antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu : 6
Cek Antrian paling belakang :
Mahasiswa paling belakang :
NIM - NAMA - PRODI - KELAS
124 - Bobi - TI - 1G

powerh...
Run: Queue...
Run: Queue...
Run: Queue...
Run: Layana...
Run: Layana...
Run: Layana...
Run: Queue...
Run: Layana...
```

Method lihatAkhir() dibuat untuk menampilkan data mahasiswa yang berada pada posisi paling belakang dalam antrian. Dalam struktur queue, elemen paling belakang ditunjukkan oleh variabel rear. Oleh karena itu, untuk menampilkan data mahasiswa terakhir, program cukup mengakses data[rear].

Sebelum menampilkan data, program melakukan pengecekan apakah antrian kosong atau tidak menggunakan method isEmpty(). Jika antrian kosong, maka akan ditampilkan pesan bahwa tidak ada data. Jika tidak kosong, maka data mahasiswa pada posisi belakang akan ditampilkan menggunakan method tampilkanData().

Selain itu, pada class utama ditambahkan menu baru yaitu pilihan ke-6 untuk memanggil method lihatAkhir(). Dengan adanya fitur ini, pengguna dapat melihat mahasiswa yang berada di urutan terakhir dalam antrian tanpa harus menampilkan seluruh isi antrian.

TUGAS

Class Diagram

Mahasiswa08
- nim : String - nama : String - prodi : String - kelas : String
tampil()

AntrianKRS08
- data : Mahasiswa08[] - front : int - rear : int - size : int - max : int - jumlahDilayani : int
+ isEmpty() + IsFull() + enqueue() + dequeue2() + lihatDepan() + lihatAkhir() + tampilSemua() + jumlahAntrian() + jumlahDilayani() + sisaBelumDilayani() + clear()

Kode program Mahasiswa08 :

```
package JOBSHEET10TUGAS;
```

```
public class Mahasiswa08 {
```

```
    String nim, nama, prodi, kelas;
```

```
    public Mahasiswa08(String nim, String nama, String prodi, String kelas) {
```

```
        this.nim = nim;
```

```

        this.nama = nama;

        this.prodi = prodi;

        this.kelas = kelas;
    }

    public void tampil() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
}

```

Kode Program AntrianKRS08 :

```

package JOBSHEET10TUGAS;

public class AntrianKRS08 {
    Mahasiswa08[] data;
    int front, rear, size, max;
    int jumlahDilayani = 0;

    public AntrianKRS08(int max) {
        this.max = max;
        data = new Mahasiswa08[max];
        front = 0;
        rear = -1;
        size = 0;
    }

    public boolean IsEmpty() {
        return size == 0;
    }
}

```

```
public boolean IsFull() {  
    return size == max;  
}
```

```
public void enqueue(Mahasiswa08 m) {  
    if (IsFull()) {  
        System.out.println("Antrian penuh!");  
    } else {  
        rear = (rear + 1) % max;  
        data[rear] = m;  
        size++;  
    }  
}
```

```
public void dequeue2() {  
    if (size < 2) {  
        System.out.println("Antrian kurang dari 2!");  
        return;  
    }  
}
```

```
System.out.println("Memproses 2 mahasiswa:");
```

```
for (int i = 0; i < 2; i++) {  
    Mahasiswa08 m = data[front];  
    m.tampil();  
    front = (front + 1) % max;  
    size--;  
    jumlahDilayani++;  
}  
}
```

```
public void lihatDepan() {  
    if (IsEmpty()) {  
        System.out.println("Kosong");  
    } else {  
        data[front].tampil();  
        data[(front + 1) % max].tampil();  
    }  
}
```

```
public void lihatAkhir() {  
    if (IsEmpty()) {  
        System.out.println("Kosong");  
    } else {  
        data[rear].tampil();  
    }  
}
```

```
public void tampilSemua() {  
    if (IsEmpty()) {  
        System.out.println("Kosong");  
    } else {  
        for (int i = 0; i < size; i++) {  
            data[(front + i) % max].tampil();  
        }  
    }  
}
```

```
public void jumlahAntrian() {  
    System.out.println("Jumlah antrian: " + size);  
}
```

```
}
```

```
public void jumlahDilayani() {  
    System.out.println("Sudah dilayani: " + jumlahDilayani);  
}
```

```
public void sisaBelumDilayani() {  
    int sisa = 30 - jumlahDilayani;  
    System.out.println("Sisa belum dilayani: " + sisa);  
}
```

```
public void clear() {  
    front = rear = -1;  
    size = 0;  
}  
}
```

Kode Program MainKRS08 :

```
package JOBSHEET10TUGAS;
```

```
import java.util.Scanner;
```

```
public class MainKRS08 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        AntrianKRS08 antrian = new AntrianKRS08(10);  
  
        int pilih;  
        do {  
            System.out.println("\n=== MENU KRS ===");
```

```

System.out.println("1. Tambah Antrian");
System.out.println("2. Proses KRS (2 orang)");
System.out.println("3. Lihat 2 terdepan");
System.out.println("4. Lihat semua");
System.out.println("5. Lihat paling belakang");
System.out.println("6. Jumlah antrian");
System.out.println("7. Jumlah dilayani");
System.out.println("8. Sisa belum dilayani");
System.out.println("0. Keluar");

System.out.print("Pilih: ");
pilih = sc.nextInt();
sc.nextLine();

switch (pilih) {
    case 1:
        System.out.print("NIM: ");
        String nim = sc.nextLine();
        System.out.print("Nama: ");
        String nama = sc.nextLine();
        System.out.print("Prodi: ");
        String prodi = sc.nextLine();
        System.out.print("Kelas: ");
        String kelas = sc.nextLine();

        antrian.enqueue(new Mahasiswa08(nim, nama, prodi, kelas));
        break;

    case 2:
        antrian.dequeue2();
        break;
}

```



```
        case 3:
            antrian.lihatDepan();
            break;

        case 4:
            antrian.tampilSemua();
            break;

        case 5:
            antrian.lihatAkhir();
            break;

        case 6:
            antrian.jumlahAntrian();
            break;

        case 7:
            antrian.jumlahDilayani();
            break;

        case 8:
            antrian.sisaBelumDilayani();
            break;
    }
} while (pilih != 0);
}
```